Cloud Computing Mini Project â€” Report

Name: Sourav Yadav Roll number: 230101043 GitHub repo: https://github.com/sourav-ydv/cloud-computing-mini-project

1. Object Storage and Parallel Teacher Labelling

Cluster created from `cluster/k3d.yaml`, SeaweedFS deployed in namespace `data`, with `datasets` and `models` buckets. `commands_pool.jsonl` uploaded to `datasets`. Teacher prompt in `prompts/teacher_prompt.txt`.

Sharding: `worker/labeler.py` reads `SHARD_INDEX` and `SHARD_COUNT` from the environment and takes every command whose line number (0-indexed) satisfies `line_number % SHARD_COUNT == SHARD_INDEX`, so two workers with `SHARD_COUNT=2` and `SHARD_INDEX=0/1` between them cover every command exactly once with no gaps or repeats (verified: 3000 unique ids across both shard outputs, 0 duplicates). Each worker writes its own object (`datasets/teacher/shard-0.jsonl`, `shard-1.jsonl`) and checkpoints every 25 records so progress survives an interruption. Both workers were run with different `SHARD_INDEX` values at the same time to demonstrate parallelism, then run to completion afterward (CPU-bound local inference made true full-length parallel execution impractical on available hardware â€” see Limitations).

Counts from `worker/clean.py` on the combined shard outputs:

Reason	Count
Input records	3000
Rejected: not valid JSON (a)	5
Rejected: wrong keys (b)	15
Rejected: bad intent (c)	11
Rejected: bad slot value (d)	65
Duplicates	48
Kept	2856

Check: 5 + 15 + 11 + 65 + 48 + 2856 = 3000. Output written to `datasets/v1/train.jsonl` with checksum in `datasets/v1/train.sha256`.

2. Training the Student

config.yaml summary: seed 42, learning rate 5.0e-5, batch size 8, 3 epochs, max sequence length 128, EleutherAI/pythia-70m, trained with PyTorch 2.3.1+cpu and Transformers 4.42.4.

Loss curve (v1, seed 42, 1500 examples): epoch 1 â€” 1.0132, epoch 2 â€” 0.6729, epoch 3 â€” 0.5260. A second version (v2, seed 123) was trained the same way: epoch 1 â€” 1.0315, epoch 2 â€” 0.6875, epoch 3 â€” 0.5471.

Model artefact locations: `models/v1/`, `models/v2/`, uploaded to the `models` bucket under `v1/` and `v2/`, each with a SHA-256 checksum written to `model.sha256` alongside the weights.

Sequence-level vs logit-based distillation

Sequence-level distillation trains the student on the teacher's final output text, treated as if it were a ground-truth label. For this project, the teacher (phi3.5, run through Ollama with temperature 0 and seed 42) generated a JSON object for each of the 3,000 commands in the pool. After cleaning, 2,856 of these teacher outputs became training examples: the student (Pythia-70M) was fine-tuned to predict the exact JSON string the teacher produced, given the command as input. The student never sees the teacher's internal computation â€” only its output text.

Logit-based distillation instead trains the student to match the teacher's full probability distribution over the vocabulary at each generation step (the "soft targets"), not just the single token the teacher happened to pick. This usually transfers more information per example, since the distribution encodes the teacher's relative confidence across all plausible next tokens, not only its top choice.

Logit-based distillation was not possible in this project because Ollama's `/api/generate` endpoint only returns the generated text, not the per-token logits or probability distribution the teacher model computed internally. Without access to those distributions, there is nothing to match the student's distribution against beyond the teacher's chosen output tokens, which is exactly what sequence-level distillation uses.

Results

Model	JSON validity	Intent accuracy	Date	Time	Place	Full-frame
Teacher (phi3.5, 3.8B)	99%	82%	84%	82%	79%	46%
Untrained Pythia-70M	0%	0%	0%	0%	0%	0%
Trained student (Pythia-70M)	99%	81%	71%	82%	79%	40%

The untrained model produced no valid JSON at all on the validation set â€” it has no notion of the task's output format. After fine-tuning on 1,500 teacher-labelled examples, the student reaches 40% full-frame accuracy, 87% of the teacher's 46%, despite being roughly 54x smaller in parameter count (70M vs 3.8B). JSON validity and intent accuracy are nearly identical between teacher and student; the main gap is in slot accuracy, particularly `date`, where the student is less consistent than the teacher.

Limitation

A specific limitation observed during generation: in a noticeable fraction of early-epoch outputs (and occasionally even in the final model), the student echoed its own prompt label â€” producing `"JSON: {...}"` or repeating `"JSON:"` a second time before the actual object â€” rather than going straight to the JSON object the teacher always produced. This is a training-data formatting artifact: with only 1,500 examples and 3 epochs on a 70M-parameter model, the student partially learned the surface pattern of the prompt template itself rather than cleanly separating "prompt" from "completion." The teacher never exhibits this behaviour. In practice this was handled by stripping a leading repeated `"JSON:"` label during post-processing before JSON parsing, which is why the reported `json_validity` (99%) is high despite this artifact â€” but it is a real limitation of the fine-tuned model's output discipline, not something hidden by the scoring method.

3. The Inference Service in a Container

All three endpoints behave as specified: /healthz returns 200 once the process is alive; /readyz returns 503 until the model finishes downloading from object storage, then 200; /parse returns the four-key JSON with X-Model-Version and X-Served-By headers set correctly.

FAULT_INJECT=1 test: /parse returned HTTP 500 with body exactly {"error": "invalid_model_output"}, as required.

Image size: see results/task3/image_size.txt â€” well under the 2GB limit (content size ~364MB).

Under --cpus 1 --memory 1g, 50 sequential /parse requests all completed with HTTP 200 (see results/task3/50_requests_status.txt). Peak memory observed was ~565MiB / 1GiB (55%), no out-of-memory kill (results/task3/peak_memory.txt).

4. Kubernetes Deployment, Scaling, and Reliability

Namespace `app`, Deployment, Service and Ingress applied and verified: `/parse` answers correctly through the Ingress at the mapped port, with the correct `X-Model-Version` and `X-Served-By` headers (see Task 3 for header format).

Resource requests/limits: cpu 250m/1, memory 512Mi/1Gi per pod. Readiness probe on `/readyz` gates traffic â€” confirmed the Service has no ready endpoints while the model is still downloading.

HPA: min 1, max 2 replicas, target 70% CPU. Observed scaling from 1 to 2 replicas under Locust load, and back down to 1 when idle (see `kubectl get hpa` output during testing).

A real bug found and fixed during testing: the first rolling-update test showed a 99% failure rate under concurrent load, with repeated pod restarts. Investigation showed the `/parse` handler was `async def` but called `model.generate()` synchronously inside it â€” this blocked FastAPI's single event loop during each ~2-3 second generation, so concurrent requests (including the liveness/readiness probes themselves) queued up and timed out. Kubernetes then killed pods it believed were unresponsive, triggering a crash-restart loop that made things worse. Fix: moved the blocking generation call into a background thread via `asyncio.to_thread`, so health checks keep responding while `/parse` requests generate. After the fix, pod restarts during load testing dropped to zero across every subsequent test run.

Rolling update: `MODEL_VERSION`/`MODEL_URI` changed from v1 to v2 while a 2-minute Locust load test ran (`--users 4 --spawn-rate 4`). Best observed run: 169 requests, 1 failure (0.59%), 0 pod restarts (`results/task4/real_test_v4_stats.csv`). A later run under colder conditions (immediately after a cold-start/pod-recovery test) showed a higher 3.70% failure rate (`results/task4/loadbalance_test_stats.csv`) â€” all failures were clean HTTP 500s from the service's own output-validation path, not crashes or timeouts, suggesting the single-CPU-per-pod generation occasionally produces malformed JSON under concurrent load rather than an infrastructure fault. Rollback via `kubectl rollout undo` confirmed working: `X-Model-Version` header correctly reverted to `v1`.

Pod recovery: deleting one pod under the Deployment showed Kubernetes automatically recreate it, restoring 2/2 Ready within seconds (`results/task4/pod_recovery.txt`).

Cold start: three runs of deleting all pods and timing until Ready gave 6s, 5s, 5s (median 5s) (`results/task4/cold_start_times.txt`).

Load balancing: with 2 Ready replicas, a 2-minute Locust run through the Ingress split successful requests 80/76 between the two pods (~51%/49%), both well above the required 30% minimum (`results/task4/loadbalance_test_stats.csv`).

Limitation: this cluster runs as a single node on one laptop. It does not demonstrate fault tolerance across multiple physical machines â€” a node failure would take down every pod regardless of replica count, since there is only one underlying host.

5. Subnetting and Network Access Control

Subnet allocation (see `network/subnets.md` for full working):

Subnet	Hosts needed	Prefix	Network	Broadcast	Usable range	Usable hosts
app	1000	/22	10.20.0.0	10.20.3.255	10.20.0.1â€“10.20.3.254	1022
public	250	/24	10.20.4.0	10.20.4.255	10.20.4.1â€“10.20.4.254	254
data	100	/25	10.20.5.0	10.20.5.127	10.20.5.1â€“10.20.5.126	126
management	20	/27	10.20.5.128	10.20.5.159	10.20.5.129â€“10.20.5.158	30

NACL (`network/nacl.json`) checked against `network/packets_sample.jsonl` with `tools/nacl_sim.py`: 6/6 packets decided correctly (100%).

A stateless NACL has no memory of connections, so inbound and outbound rules must be written separately for both directions of traffic â€” a reply to an outbound request is itself a new inbound packet as far as the device is concerned, and needs its own allow rule, or it is dropped. Rule order matters for the `198.51.100.0/24` block because NACL rules are evaluated lowest-number-first with first-match-wins: the deny rule for that range is given a lower rule number than the allow rules for ports 80/443, so a packet from that range is denied immediately regardless of which port it targets, before it can ever reach a later allow rule that would otherwise let it through.

6. Reproducibility, Testing, and CI/CD

`make up` builds and deploys the whole system from a fresh clone; `make down` tears it down.

GitHub Actions workflow (`.github/workflows/ci.yml`) runs, in order: lint (ruff) â†’ secret scan (gitleaks) â†’ unit tests (6 tests, `tests/`) â†’ model evaluation (downloads the GitHub Release model, runs `score.py`) â†’ quality gate (`score.py --min-validity 95`).

Quality gate demonstrated failing with a deliberately bad model (untrained Pythia-70M): https://github.com/sourav-ydv/cloud-computing-mini-project/actions/runs/42514219577 â€” `evaluate` succeeded (it runs regardless of model quality) but `gate` failed, since the untrained model's JSON validity was far below the 95% threshold.

`AI_USE.md` documents AI tool usage. Gitleaks reports no secrets in the repository.

7. Performance, Cost, and Review

Load test: `locust -f tools/locustfile.py --headless --users 4 --spawn-rate 4 --run-time 2m --host http://localhost:8081` (`results/task7/final_loadtest_stats.csv`).

Metric	Value
p50 latency	5100 ms
p95 latency	7600 ms
Throughput	~0.9 req/s
Error rate	3.74% (4/107)
Pod 1 CPU / memory	1002m / 656Mi
Pod 2 CPU / memory	422m / 594Mi
Cold-start (median of 3)	5s

All four failures were clean HTTP 500s from the service's own output-validation path (see Task 4), not crashes, timeouts, or resource exhaustion â€” both pods stayed within their 1 CPU / 1Gi limits throughout.

Cost

Each replica is billed for its CPU limit and memory limit every hour it runs:

```
hourly cost per replica = (1 vCPU x Rs 3.50/vCPU-hr) + (1 GiB x Rs 0.45/GiB-hr)
                        = Rs 3.50 + Rs 0.45 = Rs 3.95/hour
```

```
hourly cost, 2 replicas = 2 x Rs 3.95 = Rs 7.90/hour

requests served per hour = (103 successful requests / 120 seconds) x 3600
                         = 3090 requests/hour

cost per 1000 requests = (Rs 7.90 / 3090) x 1000 = Rs 2.56
```

SLO and error budget

SLO: 95% of requests succeed. Window: one load-test run (107 requests). Error budget = 5% x 107 = 5.35, rounded to 5 requests. Observed failures: 4. Error budget used: 4/5 = 80%. Within budget, but with little margin at this concurrency level on this hardware.

Review

Component	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	2856/3000 records kept after cleaning (95.2% yield)	NodePort used instead of port-forward after repeated connection drops during large (281MB) transfers	Use a persistent volume-backed SeaweedFS with retry-aware clients for production
Model quality	Student full-frame 40% vs teacher 46% (87% of teacher)	Student occasionally echoes its own prompt label (`"JSON:"`) before output	More training epochs or larger subset; explicit end-of-prompt token
Latency and throughput	p50 5100ms, p95 7600ms, ~0.9 req/s	Single-CPU-per-pod generation is the throughput bottleneck, not I/O	Batch multiple requests per forward pass, or allow more CPU per pod
Resource utilisation	Peak 1002m CPU / 656Mi memory per pod, within 1 CPU / 1Gi limits	One pod occasionally hit 100% of its CPU limit under load	Raise CPU limit or reduce MAX_NEW_TOKENS further
Reliability and recovery	0 pod restarts across all load tests after the blocking-generation fix; cold start median 5s	Readiness-gated rolling update dipped briefly below full capacity (Kubernetes default 25% maxUnavailable)	Set an explicit rolling update strategy with maxUnavailable: 0
Security and cost	Rs 2.56 per 1000 requests; gitleaks reports no secrets in repo	Credentials passed as dummy "any"/"any" for local SeaweedFS, not representative of real access control	Use real IAM-scoped credentials and TLS in a production deployment

Limitations and improvements

- **Single-node cluster**: the entire deployment runs on one laptop as one node. This does not demonstrate fault tolerance across multiple physical machines â€” a hardware failure takes down every pod regardless of replica count.
- **CPU-only inference**: both the teacher labelling (Ollama/phi3.5) and the student training/serving ran on CPU (teacher was GPU-accelerated via Ollama automatically on an RTX 3050; student training and the deployed service ran CPU-only per the assignment's spec). This capped throughput and made concurrent request handling the main scaling bottleneck (Task 7).
- **Network reliability**: `kubectl port-forward` proved unreliable for the ~281MB model transfer after a WiFi network change; SeaweedFS was switched to a NodePort Service for a stable connection, documented in the Task 1/3 commits.
- **A real concurrency bug was found and fixed during testing**: the inference service's `/parse` handler initially blocked FastAPI's event loop during generation, causing health-check failures and pod crash-loops under load. Fixed by moving generation into a background thread (`asyncio.to_thread`). Full detail in Task 4.
- **Improvement for future work**: logit-based distillation (matching the teacher's full probability distribution rather than just its chosen text) would likely close more of the gap between the teacher's 46% and the student's 40% full-frame accuracy, but needs a teacher that exposes logits â€” Ollama's API does not.